

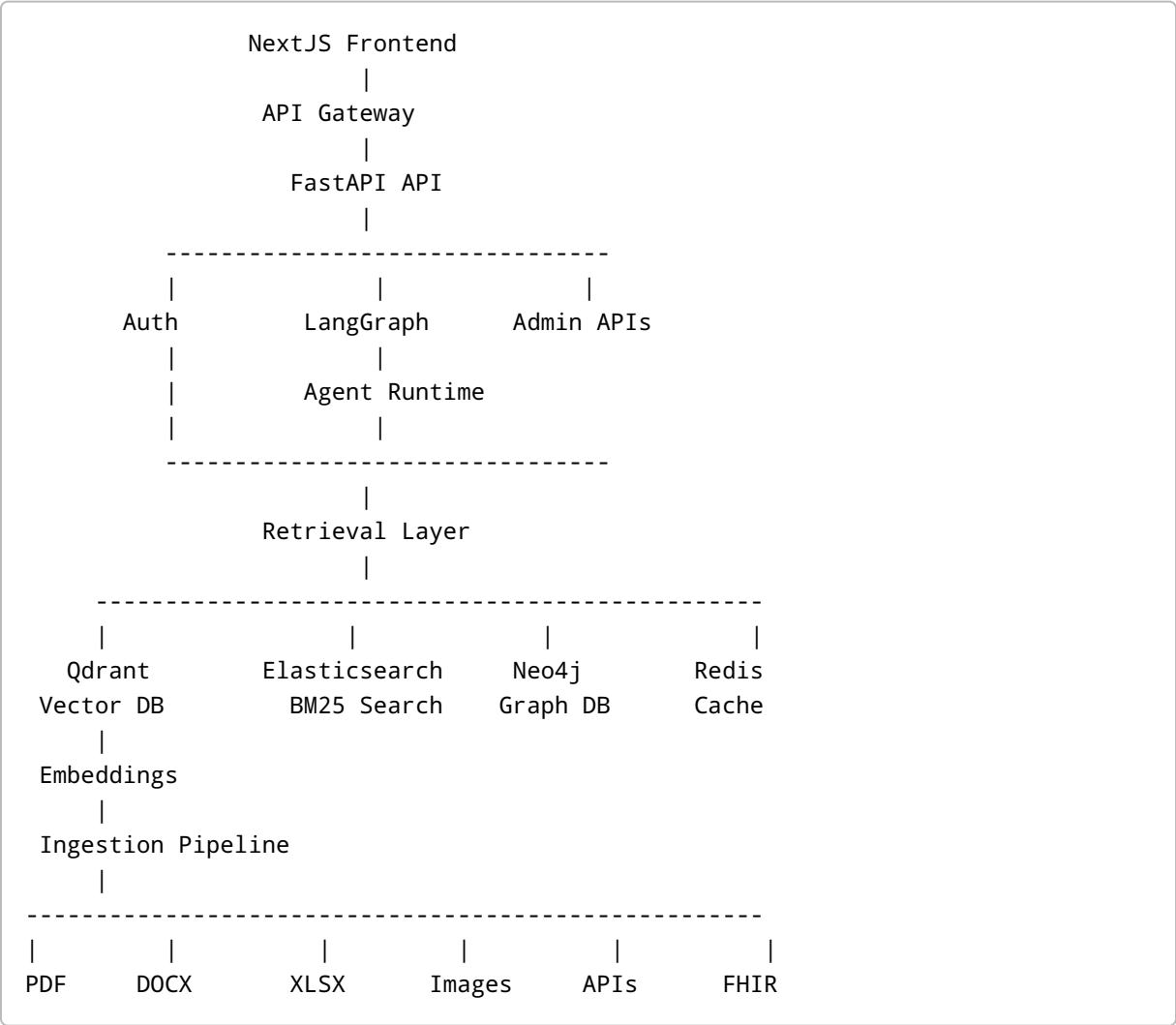
Healthcare Clinical Intelligence Platform

System Architecture Document

Version: 1.0

Status: Design Phase

1. High-Level Architecture



2. Technology Stack

Frontend

- NextJS
 - React
 - TypeScript
 - Tailwind
 - ShadCN
-

Backend

- FastAPI
 - Python 3.13
 - LangGraph
 - LiteLLM (later)
-

Databases

Supabase

Stores:

- Users
 - Organizations
 - Roles
 - Chats
 - Audit Logs
 - Evaluations
-

Qdrant

Stores:

- Embeddings
 - Retrieval Metadata
-

Neo4j

Stores:

- Medical Knowledge Graph
-

Redis

Stores:

- Cache
 - Session Data
 - Semantic Cache
-

Elasticsearch

Stores:

- BM25 Index
-

3. Multi-Tenant Architecture

Each organization has:

```
Organization
|
Users
Documents
Chats
Evaluations
Knowledge Base
```

Every object contains:

```
{
  "organization_id": "hospital_001"
}
```

No retrieval is allowed without organization filtering.

4. Document Ingestion Architecture

Supported Inputs

Documents

- PDF
- DOCX
- PPTX
- XLSX
- CSV

Images

- PNG
- JPG
- TIFF

Structured Data

- JSON
- XML
- FHIR

External Sources

- PubMed
 - WHO
 - FDA
 - CDC
-

5. Ingestion Workflow

```
Upload
|
Virus Scan
|
Document Classification
|
Content Extraction
|
Metadata Extraction
|
Chunking
```

|
Embedding Generation
|
Graph Extraction
|
Storage

6. Parsing Layer

PDF

Tool:

- Docling

Extract:

- Text
 - Tables
 - Images
 - Layout
-

OCR

Tool:

- PaddleOCR

Used for:

- Scanned PDFs
 - Images
-

Vision Processing

Tool:

- ColQwen / ColPali

Used for:

- Charts
 - Tables
 - Forms
 - Layout-heavy documents
-

7. Metadata Structure

Each chunk must contain:

```
{
  "organization_id": "",
  "document_id": "",
  "document_name": "",
  "department": "",
  "source": "",
  "document_type": "",
  "version": "",
  "roles": [],
  "created_at": ""
}
```

8. Chunking Architecture

Research Papers

Section-Based Chunking

- Abstract
 - Introduction
 - Methods
 - Results
 - Conclusion
-

SOPs

Heading-Based Chunking

- Policy
 - Procedure
 - Notes
-

Tables

One Table = One Chunk

Guidelines

Parent Child Retrieval

Parent:

Disease Guideline

Child:

Specific Recommendation

Long Documents

Semantic Chunking

9. Embedding Architecture

Text Collection

medical_text

Table Collection

medical_tables

Image Collection

medical_images

Model

Initial:

- BGE-M3

Future:

- VoyageAI
-

10. Knowledge Graph Pipeline

Extract Entities:

- Disease
- Drug
- Symptom
- Treatment
- Procedure

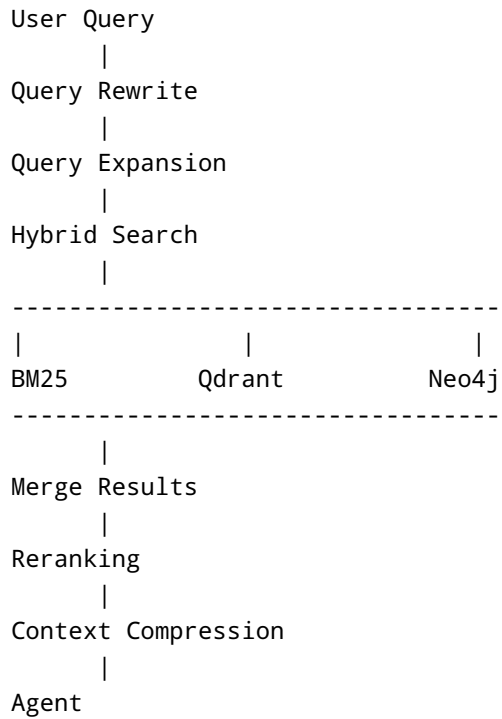
Example:

```
Metformin
  |
Treats
  |
Type 2 Diabetes
```

Stored inside Neo4j.

11. Retrieval Architecture

Retrieval Flow



12. LangGraph Agent Architecture

Planner Agent

Tasks:

- Intent Classification
- Source Selection
- Retrieval Strategy Selection

Output:

```
{
  "query_type": "clinical",
  "sources": ["pubmed", "guidelines"]
}
```

Retriever Agent

Calls:

- Qdrant
- Elasticsearch
- Neo4j
- PubMed

Verifier Agent

Checks:

- Citation Validity
- Source Agreement
- Evidence Completeness

Safety Agent

Checks:

- High Risk Advice
- Emergency Scenarios
- Pregnancy
- Pediatrics

Response Agent

Generates:

- Doctor Response
 - Research Response
 - JSON Response
-

13. Caching Architecture

Layer 1

Exact Cache

Redis

Layer 2

Semantic Cache

Redis + Embeddings

Layer 3

Retrieval Cache

Top Retrieved Chunks

Layer 4

Embedding Cache

Avoid Recomputing

14. Evaluation Architecture

Frameworks:

- RAGAS
- DeepEval

Metrics:

- Faithfulness
- Recall
- Precision
- Relevancy

- Hallucination Rate
-

15. Observability Architecture

Langfuse

Track:

- Prompts
 - LLM Calls
 - Agent Flows
-

OpenTelemetry

Trace entire request lifecycle.

Grafana

Dashboards.

Prometheus

Metrics collection.

16. Audit Architecture

Log:

- User Actions
- Upload Events
- Retrieval Events
- Agent Decisions

All stored in Supabase.

17. Security Architecture

Authentication:

- Supabase Auth

Authorization:

- RBAC

Encryption:

- TLS
- AES-256

Audit:

- Complete Query Logging

18. Deployment Architecture

Development

Docker Compose

Production

```
Cloudflare
|
Load Balancer
|
Kubernetes
|
FastAPI Pods
|
LangGraph Workers
|
-----
|         |         |         |
Qdrant Neo4j Redis Postgres
```

19. Performance Targets

- 10,000 Queries/Day
 - <5s Response Time
 - 85% Retrieval Precision
 - 95% Citation Coverage
-

20. Build Order

Phase 1

- Ingestion Pipeline
- Qdrant
- Chunking

Phase 2

- Hybrid Retrieval
- Reranking

Phase 3

- LangGraph Agents

Phase 4

- Observability

Phase 5

- Evaluation

Phase 6

- SaaS Features

Phase 7

- Production Deployment